

Lab 7 – Solutions

November 8, 2024

This lab deals with the numerical solution of a linear system

$$\mathbb{A}\mathbf{x} = \mathbf{b}, \quad \text{for } \mathbb{A} \in \mathbb{R}^{n \times n}, \mathbf{b} \in \mathbb{R}^n$$

We will focus on dynamic iterative methods.

1 Gradient method

Solving a linear system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with $\mathbb{A}$ **symmetric positive definite** is equivalent to solving the **minimization problem** of the quadratic form

$$\Phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbb{A}\mathbf{x} - \mathbf{x}^T \mathbf{b}.$$

The idea of the **preconditioned gradient method** is to compute the solution $\mathbf{x}^*$ with the sequence

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha^{(k)} \mathbb{P}^{-1} \mathbf{d}^{(k)}.$$

taking $\mathbf{d}^{(k)}$ to be the **steepest descent** direction for $\Phi(\mathbf{x})$ at $\mathbf{x}^{(k)}$ and a step length $\alpha^{(k)}$ such that $\Phi(\mathbf{x}^{(k+1)})$ is minimized. Hence these parameters are set **dynamically** to

$$\mathbf{d}^{(k)} = \mathbf{r}^{(k)} = \mathbf{b} - \mathbb{A}\mathbf{x}^{(k)}, \quad \mathbb{P}\mathbf{z}^{(k)} = \mathbf{r}^{(k)} \quad \text{and} \quad \alpha^{(k)} = \frac{\mathbf{z}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{z}^{(k)T} \mathbb{A} \mathbf{z}^{(k)}}.$$

Theorem 7.1 (Preconditioned Gradient (PG) method). *Let $\mathbb{A}$ and $\mathbb{P}$ be symmetric positive definite matrices. If*

$$\alpha^{(k)} = \frac{\mathbf{z}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{z}^{(k)T} \mathbb{A} \mathbf{z}^{(k)}}, \quad k \geq 0,$$

with $\mathbf{z}^{(k)} = \mathbb{P}^{-1} \mathbf{r}^{(k)}$ preconditioned residual, then the dynamic Richardson method $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbb{P}^{-1} \mathbf{r}^{(k)}$, is convergent for any initial guess and the convergence rate is the greatest possible.

Remark: the residual $\mathbf{r}^{(k)}$ is the **steepest descent** direction for $\Phi(\mathbf{x})$ at $\mathbf{x}^{(k)}$ and the acceleration parameters α_k are such that $\Phi(\mathbf{x}^{(k+1)})$ is minimized, with $\Phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbb{A}\mathbf{x} - \mathbf{x}^T \mathbf{b}$ (minimizing $\Phi(\mathbf{x})$ is equivalent to solve $\mathbb{A}\mathbf{x} = \mathbf{b}$.)

Preconditioned Gradient method

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbb{P}^{-1} \mathbf{r}^{(k)}$$

$$\mathbb{P}\mathbf{z}^{(k)} = \mathbf{r}^{(k)},$$

$$\alpha_k = \frac{\mathbf{z}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{z}^{(k)T} \mathbb{A} \mathbf{z}^{(k)}}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{z}^{(k)}$$

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbb{A} \mathbf{z}^{(k)} (\mathbf{b} - \mathbb{A}\mathbf{x}^{(k+1)})$$

Exercise 7.1. Consider the linear system $\mathbb{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbb{A} = \begin{bmatrix} 2 & -1 & 0 & 1 \\ -1 & 4 & 2 & -1 \\ 0 & 2 & 2 & 1 \\ 1 & -1 & 1 & 5 \end{bmatrix} \quad \text{and} \quad \mathbf{b} = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 6 \end{bmatrix}.$$

- Can you apply the gradient method to the system?
- Starting from the function for the Richardson method, implement the preconditioned gradient method in another function.
- Apply the method with no preconditioner to the system in question, starting from $\mathbf{x}^{(0)} = \mathbf{0}$ and with a tolerance of 10^{-9} .

Solution Exercise 7.1.

```
function [x, it, res, XK] = gradprec(A,b,P, x0,nmax, toll)

% Preconditioned Gradient Method
%
% INPUT:
% A: System matrix
% b: right hand side vector
% P: preconditioner
% x0: initial guess
% alpha: Acceleration Parameter
% toll: requested tolerance (relative residual)
% nmax: maximum number of iterations
%
% OUTPUT:
% x: solution
% it: number of performed iterations
% res: vector containing the normalized residual
% XK: matrix containing at each column the computed solution at k-th
% iteration

%Check dimensions
n = size(b,1);
if (size(A,1) ~= size (A,2) || size(A,1) ~= n || size(x0,1) ~= n || size (P,2) ~= n || size(P,1) ~= n )
    error('Dimesioni incompatibili')
end

% Setup
it = 0;
x = x0;
r = b - A*x;
resk = norm(r)/norm(b);
res = resk;
XK=x0;
% Iterative cycle untile the stop criterion is satisfied
while (it < nmax && resk > toll)
    it = it + 1;
    % Update the preconditioned residual by solving the linear system Pz = r.
    z = P\r;
    % Update alpha
    alpha=(z'*r)/(z'*A*z);
    % Update the solution
    x = x + alpha*z;
    XK = [XK x];
    % Update the solution
    r = b - A*x;
    resk = norm(r)/norm(b);
    res = [res; resk];
end
```

File gradprec.m

```

%% Exercise 7.1
A = [ 2 -1 0 1;
      -1 4 2 -1;
        0 2 2 1;
        1 -1 1 5];
b = [2 4 5 6]';

% The coefficient matrix is symmetric but we cannot determine from its
% structure if it is also positive definite since it is not diagonally dominant
% (notice in particular the third row).
% However, the eigenvalues of A are positive, so we can conclude that the matrix
% is positive definite and we can apply gradient method.

if all(eig(A)>0)
    fprintf("A is positive definite.\n")
end

x0 = zeros(4,1);
tol = 1e-9;
iter_max = 1000;
P = eye(size(A));
[x, iter, err] = gradprec(A,b,P, x0,iter_max, tol);
x
iter
alpha=2/(min(eig(A))+ max(eig(A)));
% Comparison with non-preconditioned richardson
[x, iter, incr] = richprec(A,b,P, alpha, x0,iter_max, tol);
% Check
x = A\b

```

Exercise 7.2. Consider the following linear systems

$$\begin{cases} 2x - y = 1 \\ -x + 2y = 1 \end{cases} \quad \begin{cases} x - y = 0 \\ -x + 20y = 19 \end{cases} \quad \begin{cases} 3x + 0y = 3 \\ 0x + 3y = 3. \end{cases}$$

- Verify that the coefficient matrices of the systems are symmetric positive definite.
- For each system graphically visualize the iterative steps of the gradient method with respect to the contour lines of the quadratic form $\Phi(\mathbf{x})$ associated to the problem

$$\Phi(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{x}^T \mathbf{b},$$

where $\mathbf{A}$ is the coefficient matrix and $\mathbf{b}$ the right-hand side of the system. What can you observe?

Solution Exercise 7.2.

```

%% Exercise 7.2
%% Point A
clc;clear;
% System 1
A = [2 -1; -1 2];
b = [1 1]';

% % System 2
% A = [1 -1; -1 20];
% b = [0 19]';
%
% % System 3
% A = [3 0; 0 3];
% b = [3 3]';

% All the matrices are symmetric and positive definite: the first and third are
% strictly diagonally dominant; for the second system we need to compute the eigenvalues
if all(eig(A)>0)
    fprintf("A is positive definite.\n")
end

```

```

%% Point B
%help meshgrid
%help contour

% Vectors for plotting the isolines
x = -2:0.01:2;
y = x;
PHI = @(x1,x2) 0.5*x1.^2*A(1,1)+...
            0.5*x2.^2*A(2,2)+...
            x1.*x2*A(1,2)-...
            x1*b(1)-x2*b(2);

% Contour
[X,Y] = meshgrid(x,y);
clear x y;
figure('Name','Iso-lines')
contour(X, Y, PHI(X,Y));

%%
P=eye(2);
x0=[-2,2]';
max_it=100; tol=1e-3;
[x1, it1, res1, XK1] = gradprec(A,b,P, x0,max_it, tol);
figure
contour(X,Y,PHI(X,Y),15);
hold on
for ii=1:it1
    x1=XK1(1,ii); x2=XK1(2,ii);
    contour(X,Y,PHI(X,Y),[PHI(x1,x2) PHI(x1,x2)]);
end
plot(XK1(1,:),XK1(2,:), '*-','linewidth',2);
fprintf("Iterations require with non preconditioned gradient Method: %g\n", it1);
% Comments
% The directions of each update are always normal to the contour lines: indeed, this is a
property of the gradient direction.
% In the first case the ellipse is relatively not stretched. Recall that the ratio
% between the lengths of the axes of the ellipses is the ratio between the maximum and minimum
% eigenvalues of the matrix (= conditioning number of the matrix). Convergence is good.
% In the second case the eigenvalues are very different and the isolines result to be very
stretched ellipses.
% The ratio between the eigenvalues is high. The gradient method is not efficient since many
steps are too short.
% In the third case the isolines are circumferences and the solution is obtained
% in just one step, since in any point on a circonference the normal goes to the center.

```

2 Conjugate gradient method

Preconditioned Conjugate Gradient method

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \tilde{\alpha}_k \mathbb{P}^{-1} \mathbf{p}^{(k)}, \quad \mathbf{p}^{(k)} \text{ s.t. } (\mathbb{A} \mathbf{p}^{(j)})^T \mathbf{p}^{(k)} = 0, i = 0 : k$$

$$\begin{aligned} \tilde{\alpha}_k &= \frac{\mathbf{p}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} \mathbb{A} \mathbf{p}^{(k)}}, \\ \mathbf{x}^{(k+1)} &= \mathbf{x}^{(k)} + \tilde{\alpha}_k \mathbf{p}^{(k)}, \\ \mathbf{r}^{(k+1)} &= \mathbf{r}^{(k)} - \tilde{\alpha}_k \mathbb{A} \mathbf{p}^{(k)} (\mathbf{b} - \mathbb{A} \mathbf{x}^{(k+1)}), \\ \mathbb{P} \mathbf{z}^{(k+1)} &= \mathbf{r}^{(k+1)}, \\ \beta_k &= \frac{(\mathbb{A} \mathbf{p}^{(k)})^T \mathbf{z}^{(k+1)}}{(\mathbb{A} \mathbf{p}^{(k)})^T \mathbf{p}^{(k)}}, \\ \mathbf{p}^{(k+1)} &= \mathbf{r}^{(k+1)} - \tilde{\beta}_k \mathbf{p}^{(k)} \end{aligned}$$

Exercise 7.3. Consider the linear system $A\mathbf{x} = \mathbf{b}$, with

$$A = \begin{bmatrix} 3 & 2 \\ 2 & 6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 2 \\ -8 \end{bmatrix} \quad \text{e} \quad \mathbf{x} = \begin{bmatrix} 2 \\ -2 \end{bmatrix}.$$

- represent the level curves of the quadratic form linked to the gradient method
Hint: to visualize the level curves of a function $f: \mathbb{R}^2 \rightarrow \mathbb{R}$, e.g. $f(x, y) = x^2 - y^2$, you can use the contour function: `contour(X,Y,f(X,Y))`
- when setting $\mathbf{x}^{(0)} = [-2, 2]^T$, how many iterations are required to reach the tolerance $\text{tol} = 10^{-6}$ by employing the gradient method? Represent the successive approximations $\mathbf{x}^{(k)}$ together with the level curves of $\Phi(\mathbf{z})$
- repeat the operations of the previous point, using the preconditioned gradient method with $P = \text{diag}(A)$;
- repeat the operations of the previous point, using the conjugate gradient method (without preconditioning) and the preconditioned version (pcg MATLAB function).
- compare all the results

Solution Exercise 7.3.

```
%% Exercise 7.3
clear; close all; clc

%% a)
A = [3 2;
     2 6];
b = [2;-8];
x_ex = [2;-2];

% Phi = 1/2 * x^T * A * x + x^T * b
Phi = @(x1,x2) 0.5*x1.^2*A(1,1)+...
           0.5*x2.^2*A(2,2)+...
           x1.*x2*A(1,2)-...
           x1*b(1)-x2*b(2);
x = linspace(-5, 5, 1001);
y = linspace(-5, 5, 1001);
[X,Y] = meshgrid(x,y);
figure
contour(X,Y,Phi(X,Y));

%% b)
x0 = [-2;2];
tol = 1e-6;
max_it = 1000;

P = eye(2);
[x1, it1, res1, XK1] = gradprec(A,b,P, x0,max_it, tol);
figure
contour(X,Y,Phi(X,Y),15);
hold on
for ii=1:it1
    x1=XK1(1,ii); x2=XK1(2,ii);
    contour(X,Y,Phi(X,Y),[Phi(x1,x2) Phi(x1,x2)]);
end
plot(XK1(1,:),XK1(2:,:), 'b--', 'linewidth', 2);
fprintf("Iterations require with non preconditioned gradient Method: %g\n", it1);
%% c)
P = diag(diag(A));
[x2, it2, res2, XK2] = gradprec(A,b,P, x0,max_it, tol);
figure
contour(X,Y,Phi(X,Y),15);
hold on
for ii=1:it2
    x1=XK2(1,ii); x2=XK2(2,ii);
    contour(X,Y,Phi(X,Y),[Phi(x1,x2) Phi(x1,x2)]);
end
plot(XK2(1,:),XK2(2:,:), 'b--', 'linewidth', 2);
fprintf("Iterations require with preconditioned gradient Method: %g\n", it2);

%% d)
[x3,it3,XK3,res3] = ConjGrad(A,b,x0,max_it,tol);
figure
```

```

contour(X,Y,Phi(X,Y),15);
hold on
for ii=1:it3
    x1=XK3(1,ii); x2=XK3(2,ii);
    contour(X,Y,Phi(X,Y),[Phi(x1,x2) Phi(x1,x2)]);
end
plot(XK3(1,:),XK3(2,:), 'x-', 'linewidth',2);
[xk_pcg1,flag1,relres1,k1] = pcg(A,b,tol,max_it,eye(2));
fprintf("Iterations require with non PCG : %g\n", k1);

%% e)
[xk_pcg2,flag2,relres2,k2] = pcg(A,b,tol,max_it,P);
fprintf("Iterations require with preconditioned gradient Method: %g\n", k2);

```

Scalar case: $f(x) = 0$, f scalar function

$$x^{(k+1)} = x^{(k)} + \delta^{(k)}, \quad \delta^{(k)} = -\frac{f(x^{(k)})}{f'(x^{(k)})},$$

where the correction $\delta^{(k)}$ is formally obtained by solving the **linear equation**

$$f'(x^{(k)})z = -f(x^{(k)})$$

Vectorial case: $\mathbf{f}(\mathbf{x}) = \mathbf{0}$, $\mathbf{f}$ vectorial function

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \delta^{(k)},$$

where the correction $\delta^{(k)}$ is found by solving the **linear system**

$$\mathbf{J}\mathbf{f}(x^{(k)})z = -\mathbf{f}(x^{(k)}), \quad \mathbf{J}\mathbf{f} \text{ Jacobian matrix of } \mathbf{f}$$

Exercise 7.4. Consider the following system of equations

$$\begin{cases} x_1^2 + x_2^2 - 1 = 0, \\ \sin(\pi x_1/2) + x_2^3 = 0. \end{cases}$$

Use the Newton method to find the solution of the system with a tolerance equal to 10^{-10} , by considering as initial guess $x_0 = -1$ and $y_0 = -1$ and $N_{max} = 200$. [Hint: use the newtonsys.m function]

Solution Exercise 7.4.

```

% EXERCISES 7.4
%% closes all figures whose handles are visible

clear
close all
clc

%Function Definition: vectorial function handle
F = @(x) [x(1).^2+x(2).^2-1;
          sin(pi*x(1)/2)+x(2).^3];
%Jacobian Definition: vectorial function handle
J = @(x) [2*x(1), 2*x(2);
          cos(pi*x(1)/2)*pi/2, 3*x(2).^2];

nmax = 200;
tol = 1e-10;
x0 = [-1;-1];

[x,res,niter,difv,x_vect] = newtonsys(F,J,x0,tol,nmax,2);
format long
x
niter

```

Exercise 7.5 (Exam Sep 6, 2024). Consider the non-linear system below, where $\mathbf{x} = [x_1, x_2]^T$ is the unknown vector:

$$\begin{cases} \frac{2x_1^2}{x_1^2 + 1} = 1 \\ \frac{2x_1^2 + 2x_2^2}{x_1^2 + x_2^2 + 1} = 1 \end{cases}$$

Exam Sep 6, 2024

1. Write the nonlinear system in the form $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x})]^T = \mathbf{0}$. Represent $\mathbf{f}$ using a Matlab® *anonymous function* that, upon receiving a column vector of 2 elements as input, returns a column vector of 2 elements containing the evaluation of $\mathbf{f}(\mathbf{x})$. Similarly, compute the Jacobian matrix $J(\mathbf{x})$ of $\mathbf{f}(\mathbf{x})$ and create the *anonymous function* that returns this matrix. Provide the MATLAB® instructions.
2. Use the provided `newtonsys.m` function to calculate the solution of the proposed nonlinear system. Use a tolerance of `tol=1e-6`, a maximum number of 1000 iterations, and the initial vector $\mathbf{x}^{(0)} = [3/2, 1/2]^T$. Then, repeat the calculation using the initial vector $\mathbf{x}^{(0)} = [-3/2, 1/2]^T$. Provide the obtained results and the number of iterations performed. Include the MATLAB® instructions used.
3. Comment on the results obtained in the previous step in light of the theory.
4. Let $e^{(k)} = \|\mathbf{x}^{(k)} - \boldsymbol{\alpha}\|$ be the error associated with the k^{th} iteration, where $\boldsymbol{\alpha}$ represent the root $(-1, 0)$. Using again the initial guess $\mathbf{x}^{(0)} = [-3/2, 1/2]^T$, construct a vector containing the errors $e^{(k)}$ obtained at each iteration. Report the MATLAB® code, as well as the values of $e^{(k)}$ for the first 4 iterations $k = 0, 1, 2, 3$ (including the initial guess). Report at least the first 3 significant figures.
5. Using the same tolerance and maximum number of iterations as above, use the `newtonsys.m` function again with $\mathbf{x}^{(0)} = [3, 1/2]^T$. Provide the obtained results and discuss them in light of the theory.

Solution Exercise 7.5.

```
clc;clear;close all;
%% 1
f=@(x) [(x(1)^2 -1) ./ (x(1)^2 + 1);
         (x(1)^2 + x(2)^2 -1) ./ (x(1)^2 + x(2)^2 + 1)];
J=@(x) [(4*x(1)) ./ (x(1)^2 + 1)^2, 0;
         (4*x(1)) ./ (x(1)^2 + x(2)^2 + 1).^2, (4*x(2))/(x(1)^2 + x(2)^2 + 1)^2];
%% 2
tol=1e-6; Nmax=1000;

x0=[1.5;0.5];
[x1,~,iter1] = newtonsys(f,J,x0,tol,Nmax,2);
fprintf("Number of iterations: %g\n", iter1)
fprintf("Solution with x0=[%g,%g]\n", x0);
fprintf("x1=%d\nx2=%d\n", x1)
%check
%fsolve(f, x0)
x0=[-1.5;0.5];

[x2,~,iter2] = newtonsys(f,J,x0,tol,Nmax,2);
fprintf("Number of iterations: %g\n", iter2)
fprintf("Solution with x0=[%g,%g]\n", x0);
fprintf("x1=%d\nx2=%d\n", x2)
%check
%fsolve(f, x0)
%% 3
% In both cases, Newton's method converges, as the number of iterations is lower than the set
maximum, and no error message is displayed.
%The nonlinear system considered has (at least) two distinct roots: [1,0]
% and [-1,0], which can be easily verified by hand.
%Depending on the initial iterate, Newton's method converges to one root rather than the
other.

%% 4
x0=[-1.5;0.5];

[~,~,iter2,~,x_vec] = newtonsys(f,J,x0,tol,Nmax,2);
alpha=[-1;0];
for ii=1:iter2
```

```

    err(ii)=norm(x_vec(:,ii)-alpha);
end
fprintf("error is:\n");
disp(err(1:4))

%% 5
x0=[3;0.5];

[x3,~,iter3] = newtonsys(f,J,x0,tol,Nmax,2);
fprintf("Number of iterations: %g\n", iter3)
fprintf("Solution with x0=[%g,%g]\n", x0);
fprintf("x1=%d\nx2=%d\n", x3)
fprintf("Newton does not converge!")
% In this case, Newton's method does not converge.
% This is not in contradiction with the theory, which states that
% Newton's method for nonlinear systems is a local method, and therefore
% only converges when the initial iterate is sufficiently close to a root of the system of
equations.
% In particular, in this case, the algorithm halts after only 8 iterations
% due to the determinant of the Jacobian matrix becoming zero
% (as indicated by the warnings printed by Matlab, which inform that the Jacobian matrix is,
% within rounding errors, a singular matrix).
```